

# NumPy (Python Numérico)

Es un [paquete](#) que nos proporciona velocidad en el cálculo matemático sobre nuestros datos.

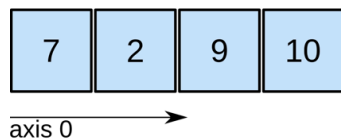
Proporciona el **Array de NumPy** como una alternativa a la **Lista de Python** regular para ejecutar operaciones elemento a elemento (es decir, elemento por elemento) entre los elementos de una secuencia de números, así como ejecutar operadores aritméticos, tales como  $+$ ,  $-$ ,  $*$  y  $/$ ; y operaciones relacionales, tales como  $>$ ,  $<$ ,  $=$  entre vectores, matrices y tensores.

## Array de NumPy

En informática, un **Array** es una estructura de datos (clase) que almacena valores de forma muy parecida a las listas, excepto que el tipo de objetos almacenados en ellos está restringido, es decir, todos los elementos deben ser del mismo tipo. La mayor ventaja de usar un array es que se puede utilizar para almacenar vectores, matrices y tensores; y por lo tanto, podemos ejecutar operaciones elemento a elemento y operaciones vector/matriz utilizando arrays.

El paquete NumPy proporciona un contenedor para la estructura de datos de array, esto se llama el [Array de NumPy](#). Los arrays pueden tener varias dimensiones; por ejemplo, un array unidimensional (array 1D) se puede utilizar para almacenar un vector con las coordenadas de un punto en el espacio 3D. En NumPy, las dimensiones también se llaman ejes. Un array bidimensional tiene dos ejes correspondientes: el primero corre verticalmente hacia abajo a través de las filas (eje 0), y el segundo corre horizontalmente a través de las columnas (eje 1).

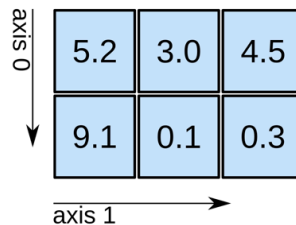
## 1D array



shape: (4,)

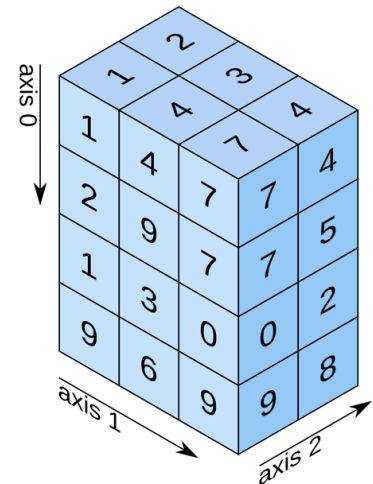
\newline

## 2D array



shape: (2, 3)

## 3D array



shape: (4, 3, 2)

La representación computacional del vector es, `array([7, 2, 9, 10])`, tiene solo un eje.

Además, este eje tiene 4 elementos en él, así que decimos que tiene una longitud de 4. En el siguiente ejemplo, queremos almacenar una matriz (2 dimensiones) en un array, por lo que el array tiene 2 ejes. El primer eje (eje 0) tiene una longitud de 2, el segundo eje (eje 1) tiene una longitud de 3. Este array tiene 6 componentes.

*Matrix : \*MathematicalRepresentation*

$$A_{m,n} = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{bmatrix}$$

*Example*

$$\begin{bmatrix} 5.2 & 3.0 & 4.5 \\ 9.1 & 0.1 & 0.3 \end{bmatrix}$$

*\*ComputationalRepresentation*

`array([[5.2, 3.0, 4.5], [9.1, 0.1, 0.3]])`

## Tensor - array nD

Un tensor es un array de objetos matemáticos (usualmente números o funciones) que se transforma de acuerdo a ciertas reglas bajo cambio de coordenadas. En un espacio  $nD$  (ej. 2D y 3D), un tensor de rango- $k$  tiene  $n^k$  componentes que pueden ser especificados con referencia a un sistema de coordenadas dado. En consecuencia, un escalar, tal como la temperatura, es un tensor de rango-0 con (asumiendo un espacio 3D)  $3^0 = 1$  componente, un vector, tal como la fuerza, es un tensor de rango-1 con  $3^1 = 3$  componentes, y el estrés

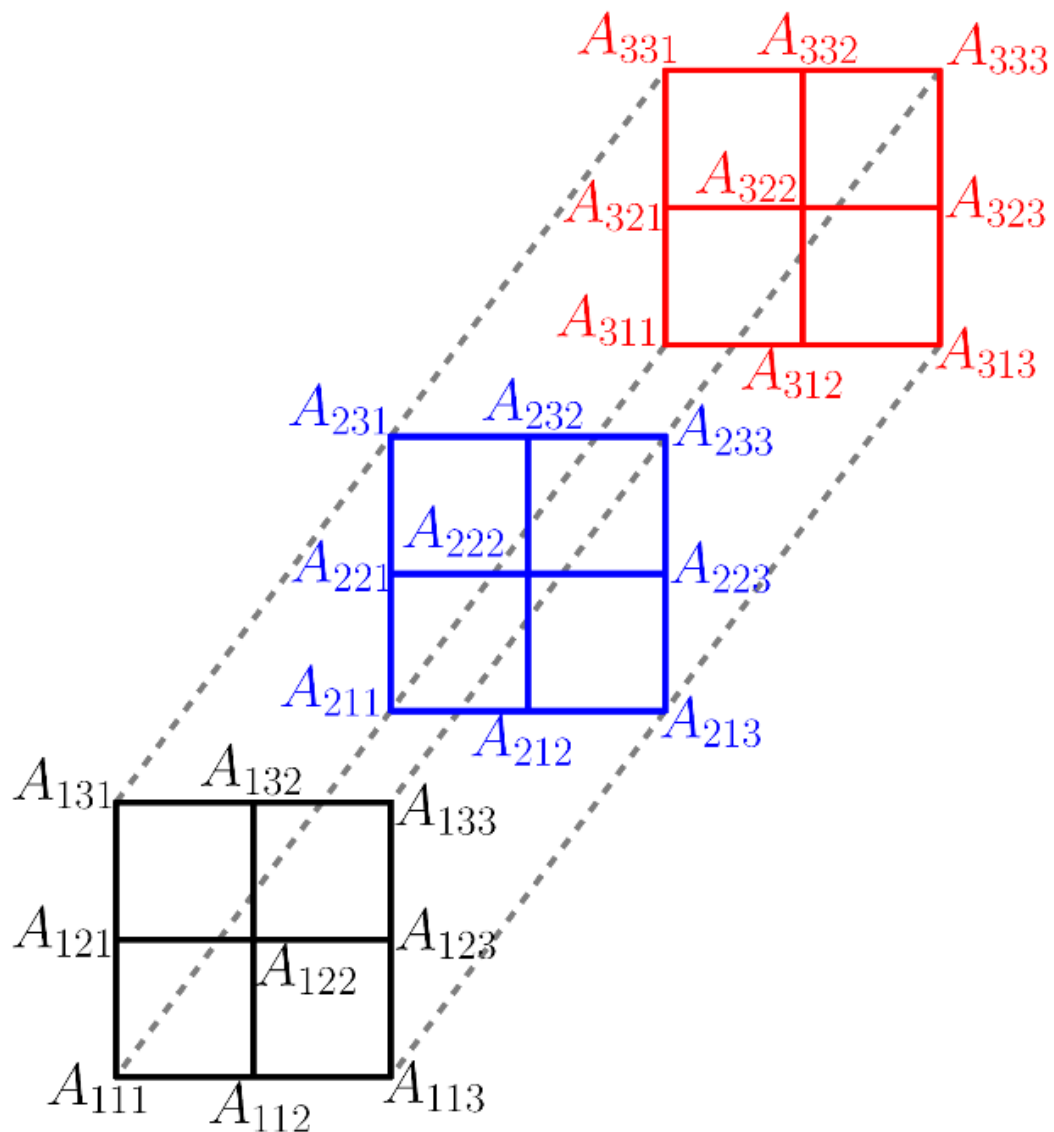
es un tensor de rango-2 con  $3^2 = 9$  componentes. Así que el rango es independiente del número de dimensiones espaciales (nD).

Notas: El estrés es una cantidad física que expresa las fuerzas internas que las partículas vecinas de un material continuo ejercen entre sí

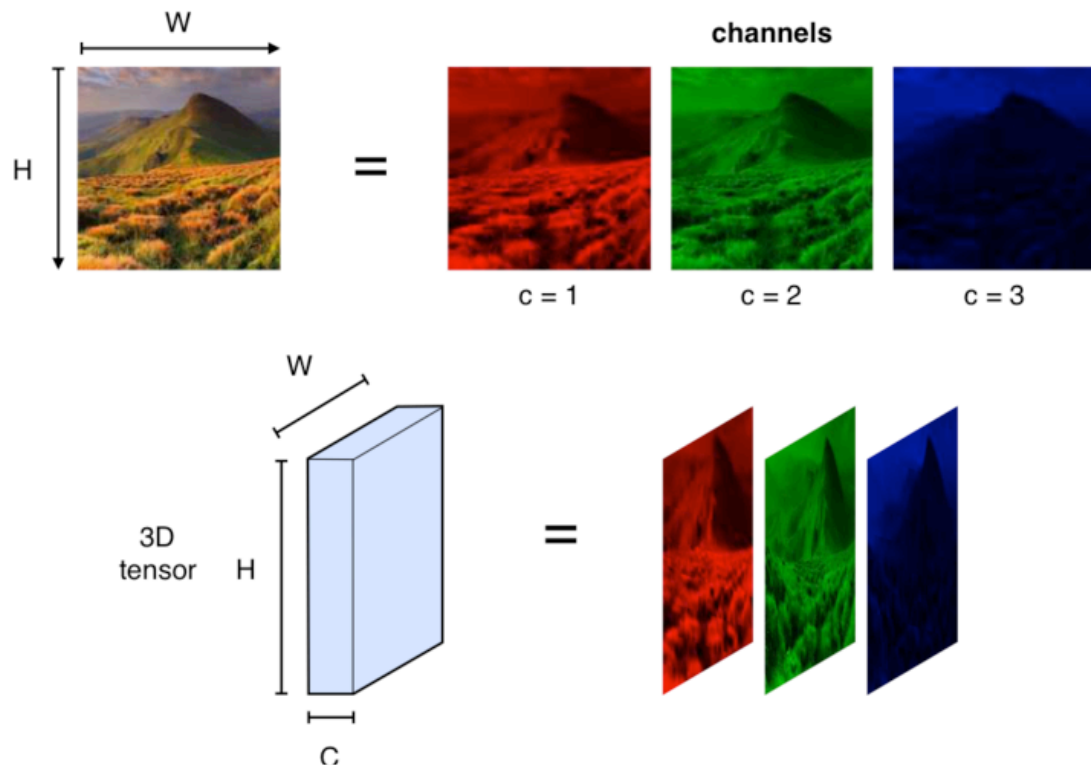
Una estructura muy común es el tensor de rango-2, una matriz de  $3 \times 3$  que representa el mapa de una cosa 3D en otra cosa 3D. La "Relatividad General" utiliza muchos tensores de rango-2 en 4 dimensiones de espacio-tiempo, representados por una matriz de  $4 \times 4$ .

En la siguiente figura se ilustra gráficamente la estructura de un tensor de rango-3 en un espacio 3D. (Tomado de Taha Sochi, Principles of tensor calculus).

**PREGUNTA:** ¿Cuáles son las longitudes de los ejes de este tensor?



Un tensor de rango 3 en un espacio 3D (un cubo de 3x3x3 dimensiones). por lo tanto la longitud de sus ejes es 3,3,3.



## Importando un Array de NumPy

Para usar un Array de Numpy primero necesitamos importar el Paquete Numpy. Lo importamos como un objeto llamado "np". Usualmente este objeto se llama "np" pero puede tener cualquier nombre.

```
In [1]: # Usually, we first create a "np" object of the numpy package.
import numpy as np
```

## Creando un Array de NumPy

Para almacenar un vector (array unidimensional), una matriz (array bidimensional) o un tensor (array n-dimensional); necesitamos crear un objeto de la clase array de NumPy llamado ndarray. <class 'numpy.ndarray'> (Array N-dimensional). También se conoce como "array".

```
In [2]: #Let's create an array object that stores a vector
#vector_object = np.array(1,2,3,4)    # WRONG
#In order to create an array you need to provide lists
new_array = np.array([1,2,3,4])    # RIGHT
```

```
print(type(new_array))  
new_array
```

```
<class 'numpy.ndarray'>
```

```
Out[2]: array([1, 2, 3, 4])
```

## Cómo encapsular (o almacenar) ya sea un vector o una matriz dentro de un Array

```
In [3]: vector_object = np.array([1,2,3,4])  
print(type(vector_object))  
vector_object
```

```
<class 'numpy.ndarray'>
```

```
Out[3]: array([1, 2, 3, 4])
```

```
In [4]: matrix_object = np.array([[1,0,0],[0,1,2]])  
matrix_object
```

```
Out[4]: array([[1, 0, 0],  
               [0, 1, 2]])
```

```
In [5]: #We want to check the dimensions of the previous vector and matrix  
print(vector_object.ndim)  
print(matrix_object.ndim)
```

```
1  
2
```

Se espera que un vector tenga una dimensión igual a 1, y una matriz una dimensión igual a 2

```
In [6]: #Now Let's figure out the length of the axes  
print(vector_object.shape)  
print(matrix_object.shape)
```

```
(4,)  
(2, 3)
```

## Conceptos matemáticos de matrices

**PREGUNTA:** ¿Cuál es la diferencia entre la multiplicación elemento a elemento y la multiplicación de matrices?

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 \\ b_4 & b_5 & b_6 \\ b_7 & b_8 & b_9 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 \\ c_7 & c_8 & c_9 \end{bmatrix}$$

### Multiplicación elemento a elemento

$$c_1 = a_1 b_1$$

$$c_2 = a_2 b_2$$

### Multiplicación de matrices

$$c_1 = a_1 b_1 + a_2 b_4 + a_3 b_7$$

$$c_2 = a_1 b_2 + a_2 b_5 + a_3 b_8$$

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{in}b_{nj} = \sum_{k=1}^n a_{ik}b_{kj}$$

- Multiplicación elemento a elemento: Se multiplican los elementos que están en la misma posición en ambas matrices. El resultado es una matriz del mismo tamaño que las originales. Ejemplo:  $c_{ij} = a_{ij} \times b_{ij}$
- Multiplicación de matrices: Se multiplica la fila de la primera matriz por la columna de la segunda matriz y se suman los resultados (producto escalar). Para poder multiplicarlas, el número de columnas de la primera debe ser igual al número de filas de la segunda. Ejemplo:  $c_{ij} = \sum a_{ik} \times b_{kj}$  (fila por columna)

### Las Listas de Python no pueden ejecutar operaciones elemento a elemento

Para ilustrarlo, intentemos calcular el índice de masa corporal (IMC) para varias personas

$$\text{BMI} = \frac{\text{weight}_{\text{kg}}}{\text{height}_{\text{m}}^2}$$

```
In [7]: #height is a list with the height of several people
height = [1.74, 1.56, 1.67, 1.69]
# mass or weight
weight = [80, 55.6, 65.5, 70.6]
```

```
#Let's try to calculate the BMI for all of the people just by doing arithmetic oper
BMI_calculation = weight / height **2
```

```
-----
TypeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_16180\3101344131.py in <module>
      4 weight = [80.3, 55.6, 65.5, 70.6]
      5 #Let's try to calculate the BMI for all of the people just by doing arithmet
ic operators between lists
----> 6 BMI_calculation = weight / height **2

TypeError: unsupported operand type(s) for ** or pow(): 'list' and 'int'
```

La salida devuelve TypeError: unsupported operand type(s) for \*\* or pow(): 'list' and 'int'

Esto significa que no somos capaces de hacer este cálculo de esa manera. Probablemente necesitaremos hacer algo como esto:

```
In [8]: %%time
height = [1.73, 1.56, 1.67, 1.69]
# mass or weight
weight = [80.3, 55.6, 65.5, 70.6]

bmi_list = []
for i in range(4):
    bmi_list.append(weight[i] / (height[i]**2))

print(bmi_list)

[26.830164723178186, 22.846811308349768, 23.485962207321883, 24.719022443191765]
Wall time: 0 ns
```

## Conceptos de Array

### Vectorización

La vectorización aplica eficientemente operaciones a grupos de elementos. Usando vectorización podemos ejecutar operaciones elemento a elemento

A diferencia de las listas, los arrays de numpy pueden ejecutar operaciones elemento a elemento, tales como operaciones aritméticas (+, -, \*, /) y relacionales (>, <, ==, !=) entre arrays e incluso con valores numéricos.

Primero necesitamos convertir las listas a arrays.

```
In [3]: %%time
height = [1.73, 1.56, 1.67, 1.69, 1.66]
# mass or weight
weight = [80.3, 55.6, 65.5, 70.6, 62]
#Create the arrays
height_array = np.array(height)
```

```
weight_array = np.array(weight)

#Aritmethic operarion between an array and a numerical value
square_height = height_array ** 2

#Arithmetic operation among arrays
bmi = weight_array / height_array ** 2
bmi
#print(bmi)
```

CPU times: total: 0 ns

Wall time: 0 ns

Out[3]: array([26.83016472, 22.84681131, 23.48596221, 24.71902244, 22.4996371 ])

## Indexación (Indexing)

Indexar un array devuelve elementos individuales, subarrays o elementos que satisfacen una condición específica.

### Indexación para copiar, con máscara

**EJERCICIO:** Crea un array booleano de numpy donde cada elemento del array debe ser True si el IMC de la persona correspondiente está por debajo de 23. Puedes usar el operador < para esto. Nombra el nuevo array como "light".

In [11]: light = bmi < 23

In [12]: print(light)

[False True False False True]

### Indexación para visualizar, con máscara

Ahora, imprimamos un array de numpy con los IMCs de solo las personas cuyo IMC está por debajo de 23. Usamos "light" dentro de corchetes para hacer una selección en el array "bmi". Podemos usar corchetes para subconjuntos de arrays de numpy.

In [13]: print(bmi[light])

[22.84681131 22.4996371 ]

## Subconjuntos de Arrays de Numpy (Indexación)

Definamos una matriz  $A$ , donde queremos obtener el valor  $A_{23}$

$$A = A_{m,n} = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{bmatrix}$$

$$A = A_{m,n} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 2 \end{bmatrix}$$



Recuerda que en los Arrays de Numpy el índice del primer elemento es 0, y el último índice es  $N - 1$ , donde  $N$  es la longitud de los ejes.

```
In [5]: A = np.array([[1,0,0],[0,1,2]])
print(A)
print(A[1][2])
#As alternative we can also get that value with this syntax
print(A[1,2])

[[1 0 0]
 [0 1 2]]
2
2
```

## Rebanado y troceado (Slicing)

### Subconjuntos de varios elementos a la vez

```
subset = my_array[row, start_column:end_column]
```

```
subset = my_array[start_row:end_row, column]
```

```
subset = my_array[start_row:end_row, start_column:end_column]
```

```
In [8]: #We can also get an entire row
R1 = A[0,0:2]
R1
```

```
Out[8]: array([1, 0])
```

```
In [7]: #Alternative code to get an entire row
R1 = A[0,:]
R1
```

```
Out[7]: array([1, 0, 0])
```

**EJERCICIO:** Haz un subconjunto de la matriz A para obtener esta matriz B de  $2 \times 2$

$$B = B_{m,n} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

```
In [4]: import numpy as np
A = np.array([[1, 0, 0],
              [0, 1, 2]])
# Crear el subconjunto B (2x2)
B = A[:2, :2]
print("Matriz A:")
print(A)
print("\nMatriz B (subconjunto):")
print(B)
```

Matriz A:

```
[[1 0 0]
 [0 1 2]]
```

Matriz B (subconjunto):

```
[[1 0]
 [0 1]]
```

**EJERCICIO 2:** Haz un subconjunto de la matriz A para obtener esta nueva matriz C:

$$C = C_{m,n} = \begin{bmatrix} A_{13} \\ A_{23} \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

```
In [5]: import numpy as np

A = np.array([[1, 0, 0],
              [0, 1, 2]])

C = A[:, 2:3]
print("Matriz C:")
print(C)
```

Matriz C:

```
[[0]
 [2]]
```

## Reducción de Arrays de Numpy (Reducción)

Definamos una matriz A, donde queremos obtener la suma de todos los valores en cada columna. Es decir:

$$A = A_{m,n} = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{bmatrix}$$

$$S = S_n = [A_{11} + A_{21} \quad A_{12} + A_{22} \quad A_{13} + A_{23}]$$

Las operaciones de reducción actúan a lo largo de uno o más ejes. En este ejemplo, un array se suma a lo largo del eje 0 para producir un vector.

```
In [21]: A = np.array([[1,0,0],[0,1,2]])
S = sum(A)
print(S)
```

```
[1 1 2]
```

## Difusión (Broadcasting)

Exploremos la difusión en la multiplicación elemento a elemento de arrays bidimensionales.

$$B \circ C$$

en código `B * C`.

$$B \circ C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \circ \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

In [22]: `B*C`

```
-----
NameError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_16180\2428265818.py in <module>
----> 1 B*C

NameError: name 'B' is not defined
```

## Multiplicación de Matrices

Exploremos una multiplicación de matrices

$$B \cdot C$$

en código `B @ C`.

$$B \cdot C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

In [ ]: `B @ C`

**Pregunta:** ¿Cuál es la diferencia entre difusión (broadcasting) y multiplicación de matrices?

La diferencia principal radica en si se requiere o no que las dimensiones coincidan exactamente y en cómo se alinean los datos para la operación:

**Difusión (Broadcasting):** No es una operación matemática per se, sino una regla de NumPy para permitir operaciones (como suma o multiplicación elemento a elemento) entre arrays de diferentes formas. NumPy "estira" o duplica virtualmente el array más pequeño a lo largo de las dimensiones faltantes para que coincida con el tamaño del array más grande. Ejemplo: Sumar un vector columna (3, 1) a una matriz (3, 3). El vector se "copia" a todas las columnas.

**Multiplicación de matrices:** Es una operación algebraica específica (producto punto). Requiere una coincidencia estricta de las dimensiones internas: Para multiplicar  $A \times B$ , el número de columnas de  $A$  debe ser igual al número de filas de  $B$ . No implica "estirar" datos; combina filas y columnas mediante suma de productos.